Movie Booking Backend - Student Guide

Node.js + MongoDBClient | Routes, RBAC, Session and Running Steps

1. Application Running Steps

Install dependencies

```
npm install
```

Seed MongoDB data

```
npm run seed
```

Start server

```
npm start
```

Save a `MOVIE_ID` printed or returned from the movie data.

Base URL: `http://localhost:3000`

Seed users:

```
User
Email: user@test.com
Password: user123
```

```
Theater
Email: theater@test.com
Password: theater123
```

---

2. Routes to Build

Route 1 - Login

POST /login

- Read email and password.
- Find the user in the `users` collection.
- Verify the password.
- Create a session token.
- Store `userId`, `role` and `expiresAt` in the session.
- Return token and role.
- Expected: `200` success, `401` invalid credentials.

```
{
  "email": "user@test.com",
  "password": "user123"
}
```

Route 2 - Logout

POST /logout

- Requires a valid session.
- Delete the current session token.
- Expected status: `200`.

Route 3 - View Movies

GET /movies

- Requires a valid session.
- Fetch movies from the `movies` collection.
- Return movie information and show times.
- Expected status: `200`.

Route 4 - Book Seat

POST /bookings

- Role required: `user`.
- Validate `movieId`, `date`, `showTime` and `seatNo`.

- Find the requested movie.
- Verify that the requested show time exists for that movie.
- Insert booking with:
- `userId`
- `movieId`
- `date`
- `showTime`
- `seatNo`
- `bookedAt`
- Prevent duplicate booking for the same movie, date, show time and seat.
- Expected: `201` created, `400` invalid input, `404` movie not found, `409` seat already booked.

```
{
  "movieId": "MOVIE_ID",
  "date": "2026-08-25",
  "showTime": "10:00",
  "seatNo": "A5"
}
```

Route 5 - View Logged-In User Bookings

GET /bookings

- Role required: `user`.
- Match `userId` with the logged-in user ID.
- Return only that user's bookings.
- Use `find()` and `toArray()`.
- Do not use aggregation.
- Expected status: `200`.

Route 6 - View Theater Movies

GET /theater/movies

- Role required: `theater`.
- Match `theaterId` with the logged-in theater ID.
- Return movies belonging to that theater.
- Expected status: `200`.

Route 7 - Add Movie

POST /movies

- Role required: `theater`.
- Read title, language, duration and show times.
- Store logged-in theater ID as `theaterId`.
- A movie must contain exactly 4 unique show times.
- Insert the movie into the `movies` collection.
- Expected: `201` created, `400` invalid input.

```
{
  "title": "Avatar",
  "language": "English",
  "duration": 162,
  "showTimes": [
    "10:00",
    "13:00",
    "16:00",
    "20:00"
  ]
}
```

Route 8 - Update Movie

PUT /movies/:movieId

- Role required: `theater`.
- Validate `movieId`.
- Allow updating:
- title
- language
- duration
- showTimes
- If show times are updated, require exactly 4 unique values.
- Match both `movieId` and logged-in `theaterId`.
- Use `updateOne()`.
- Expected: `200` updated, `400` invalid input, `404` movie not found.

```
{
  "title": "Avatar Updated",
  "showTimes": [
    "09:00",
    "12:00",
```

```
    "16:00",
    "21:00"
  ]
}
```

Route 9 - Delete Movie

DELETE /movies/:movieId

- Role required: `theater`.
- Validate `movieId`.
- Match both movie ID and logged-in theater ID.
- Delete the movie using `deleteOne()`.
- Delete bookings related to that movie using `deleteMany()`.
- Expected: `200` deleted, `400` invalid movie ID, `404` movie not found.

---

3. Session Guidelines

- Store sessions in an in-memory `Map`.
- Generate session tokens using `crypto.randomBytes()`.
- Session duration: **30 minutes**.
- Each session stores:

```
userId
role
expiresAt
```

Set:

```
expiresAt = Date.now() + 30 minutes
```

For protected requests, send:

```
Authorization: Bearer <SESSION_TOKEN>
```

For every protected request:

1. Read the `Authorization` header.
2. Extract the Bearer token.
3. Find the token in the session Map.

4. Check session expiry.
5. Delete the session if expired.
6. Return `401` if the session is missing or expired.

Protected routes:

```
POST /logout
GET /movies
POST /bookings
GET /bookings
GET /theater/movies
POST /movies
PUT /movies/:movieId
DELETE /movies/:movieId
```

Public route:

```
POST /login
```

---

4. RBAC Guidelines

Two roles are available:

```
user
theater
```

User Role

Allowed:

```
GET /movies
POST /bookings
GET /bookings
POST /logout
```

Not allowed:

```
GET /theater/movies
POST /movies
```

```
PUT /movies/:movieId
DELETE /movies/:movieId
```

Theater Role

Allowed:

```
GET /movies
GET /theater/movies
POST /movies
PUT /movies/:movieId
DELETE /movies/:movieId
POST /logout
```

Not allowed:

```
POST /bookings
GET /bookings
```

Use:

```
requireAuth()
```

for routes accessible to any logged-in role.

Use:

```
requireRole()
```

for role-specific routes.

Return:

```
401 - Missing or invalid session
403 - Role not allowed
```

---

5. Booking Constraint

Create a unique MongoDB index using:

```
movieId
date
showTime
seatNo
```

The same seat cannot be booked twice for the same:

```
Movie + Date + Show Time + Seat
```

If MongoDB returns duplicate key error:

```
error.code === 11000
```

return:

```
409
```

---

6. Final Route List

```
POST   /login
POST   /logout
GET    /movies
POST   /bookings
GET    /bookings
GET    /theater/movies
POST   /movies
PUT    /movies/:movieId
DELETE /movies/:movieId
```

Do not implement CORS.

Do not use MongoDB aggregation.